



Low-discrepancy sequence initialized particle swarm optimization algorithm with high-order nonlinear time-varying inertia weight

Chengwei Yang, Wei Gao*, Nengguang Liu, Chongmin Song

The School of Civil and Environmental Engineering, The University of New South Wales, Sydney, NSW 2032, Australia

ARTICLE INFO

Article history:

Received 5 November 2013

Received in revised form 6 January 2015

Accepted 6 January 2015

Available online 20 January 2015

Keywords:

Low-discrepancy sequence

Particle swarm optimization

$1/\pi^2$ order nonlinear function

Nonlinearly decreasing inertial weight

Constant acceleration coefficients

ABSTRACT

Particle swarm optimization (PSO) is a population-based stochastic optimization algorithm motivated by intelligent collective behavior of some animals such as flocks of birds or schools of fish. The most important features of the PSO are easy implementation and few adjustable parameters. A novel PSO method called LHNPSO, with low-discrepancy sequence initialized particles and high-order ($1/\pi^2$) nonlinear time-varying inertia weight and constant acceleration coefficients, is proposed in this paper. The initial population particles are generated by using the Halton sequence to fill the search space efficiently. Nonlinear functions with orders varied within big ranges are employed to adjust the inertial weight, cognitive and social parameters. Based on the sensitivity analysis of PSO performance to the changes of the orders of these nonlinear functions, $1/\pi^2$ order nonlinear function is selected to adjust the time-varying inertia weight and the two acceleration coefficients are set to be constants. A set of well-known benchmark optimization problems is then used to investigate the performance of the proposed LHNPSO algorithm and facilitate the comparison with other three types of PSO algorithms. The results show that the easily implemented LHNPSO can converge faster and give a much more accurate final solution for a variety of benchmark test functions.

© 2015 Elsevier B.V. All rights reserved.

1. Introduction

With the inspiration motivated by the research results on the modeling and the simulations of the behavior reflected in flocks of birds, Kennedy and Eberhart proposed the particle swarm optimization (PSO) algorithm [1]. The algorithm is a stochastic population-based method and is regarded as a global search strategy. In the PSO, particles move through the problem space with a specified velocity in search of optimal solution. Each particle maintains a memory which helps it keep the track of its previous best position and the global best position. The most important advantages of PSO are that it is easy to implement and has few parameters to adjust [2]. Commonly, the adjustable parameters are inertial weight, cognitive and social parameters.

PSO algorithm has attracted considerable attention and has been used in many research areas over the past decades [3–11]. Generally, convergence speed and ability to find global optima are basic criteria for assessing the performance of an optimization method. A large number of improved PSO algorithms [12–22] have been

proposed to achieve the two goals, that is, faster convergence speed and avoiding premature or local optima.

The convergence speed of a PSO algorithm is dependent on the inertia weight and two acceleration coefficients. Random [23], linear time-varying [24], nonlinear time-varying [25,26] and adaptive strategies [27–29] have been used to adjust the inertia weight to improve the convergence speed. A review and summary of various inertia weight modification mechanisms reported in literature can be found in reference [2]. Nonlinear time-varying and adaptive inertia weights normally have better performance than others. Moreover, the principle and implementation of nonlinearly decreasing inertia weights are simpler than those of adaptive ones. Currently, third and less order nonlinear functions have been investigated extensively for adjusting inertia weight, however, the performance of higher order nonlinear time-varying weight needs to be further studied.

The concept for initializing the particles of most PSO algorithms is same as that for generating random numbers in the traditional Monte-Carlo simulation method which could be regarded as the original stochastic optimization method. In contrast to traditional Monte-Carlo methods using pseudo random numbers, the quasi-Monte Carlo method produces deterministic sequences of well-chosen points that provide the best-possible spread in the change ranges of variables [30]. These deterministic sequences are

* Corresponding author. Tel.: +61 293854123; fax: +61 293856139.

often referred to as low-discrepancy sequences filling the sample area efficiently and uniformly [31], which have been successfully used to solve globally optimal problems [30–33].

The major objective of this paper is to propose an improved PSO algorithm to benefit by the advantages of high-order nonlinear time-varying inertia weight and low-discrepancy sequence. This paper is organized as follows. Section 2 reviews the classical PSO and some commonly accepted improved version of PSO. In Section 3, different-order nonlinear functions, varying from 1/18 to 9, are tested to adjust the inertia weight and two acceleration coefficients, and the corresponding parametric studies on the orders are also implemented. The improved PSO with adjusted coefficients are further test with different population sizes. Section 4 presents an improved PSO method with low-discrepancy sequence initialized particles and high-order $(1/\pi^2)$ nonlinear dynamic varying inertia weight (LHNPSP), and experimental results by using well-known benchmark test functions. Section 5 gives a brief conclusion about this study.

2. Review of classical PSO and its variants

2.1. PSO

PSO algorithm starts with a population of particles randomly initialized in the search space. Each particle represents a potential solution. The algorithm searches the optimal solution by moving the positions of particles in the search space. The position and velocity of the i th particle are represented by n -dimensional vector $x_i = (x_{i1}, x_{i2}, \dots, x_{in})$ and $v_i = (v_{i1}, v_{i2}, \dots, v_{in})$, respectively. The particle moves its current position toward the global optimum based on two items, that is, the best position encountered by this particle and the best position visited by the whole swarm. The velocity and position of the particle are updated according to the following formulations

$$v_i(k+1) = wv_i(k) + c_1 r_1 \circ (x_i^{pb} - x_i(k)) + c_2 r_2 \circ (x^{Gb} - x_i(k)) \quad (1)$$

$$x_i(k+1) = x_i(k) + v_i(k+1) \quad (2)$$

where k is the iteration number, $v_i(k)$ denotes the velocity of the i th particle at the k th iteration, $x_i(k)$ represents the position vector of the i th particle at the k th iteration, vector x_i^{pb} is the best position visited by the i th particle, vector x^{Gb} is the global best location found by the whole swarm until the current iteration. w is the inertia weight controlling the influence of the previous velocity on the current one. In this paper, the maximum velocity is 20% of the search range divided by the size of particles. c_1 is the cognitive parameter, and c_2 is the social parameter. The two acceleration coefficients and represent dependent settings which indicate the degree of confidence in the best solution found by the individual particle and by the entire swarm, respectively. r_1 and r_2 are two random vectors consist of random numbers uniformly distributed in the interval $[0, 1]$. The symbol $\circ$ in Eq. (1) denotes the Hadamard product.

2.2. LPSO and LPSO-TVAC

The global convergence of a PSO algorithm is dependent on the degree of local/global exploration controlled by the two acceleration coefficients, meanwhile, the relative rate of convergence is affected by the inertia weight parameter. Research results have shown that for a fixed/constant inertia value there is a significant reduction in the algorithm convergent rate. In the earlier optimization stage, a large inertia weight is required in order to search the design space thoroughly. When the most promising areas of the design space have been discovered and the convergence rate starts to slow down, the inertia weight should be reduced, in order for the particles' momentum to decrease allowing them to concentrate in

the best design areas. Therefore, Shi and Eberhart [34] proposed an improved PSO algorithm with a linear time-dependent value of the inertia weight (LPSO) to accomplish the aforementioned strategy

$$w(k+1) = w_{\max} - \frac{w_{\max} - w_{\min}}{k_{\max}} \cdot k \quad (3)$$

where k is the iteration number starting from iteration zero; $k_{\max}$ is the maximum number of allowable iteration; $w_{\max}$ and $w_{\min}$ are the maximum and minimum values of the inertia weight, respectively. Usually the value of the inertia weight varies between 0.4 and 0.9.

Similarly, the PSO algorithm with linearly decreasing inertia weight and time-varying acceleration coefficients (LPSO-TVAC) has been proposed by Ratnaweera et al. [35]. In the early stage, a large c_1 and small c_2 allow the particles to move around the whole search space instead of moving toward the population best. In the latter stage, a small c_1 and a large c_2 allow the particles to converge into the global optimum. c_1 and c_2 are computed by

$$c_1(k+1) = c_{1i} - \frac{c_{1i} - c_{1f}}{k_{\max}} \cdot k \quad (4)$$

$$c_2(k+1) = c_{2i} - \frac{c_{2i} - c_{2f}}{k_{\max}} \cdot k \quad (5)$$

where c_{1i} , c_{1f} , c_{2i} and c_{2f} are initial and final values of the cognitive and social parameters, respectively. The best reported results were achieved when $c_{1i} = c_{2f} = 2.5$ and $c_{1f} = c_{2i} = 0.5$ [20].

Although numerous improved PSO algorithms have been presented and applied in many research areas, solving optimization problems with high accuracy and rapid convergence speed is still an important task. Both LPSO and LPSO-TVAC achieve better convergence and accuracy than classical PSO, and also keep the developed formulations structurally simple and easily understandable, making these variants become handy tools to solve optimization problems in a wide range of subjects. In other words, these two variants preserve one of the major merits of PSO, easy implementation. Because of this, in this paper, we manage to further improve PSO based on the ideas from LPSO and LPSO-TVAC, which will be explained at a greater extent in the next section. PSO, LPSO and LPSO-TVAC are all adopted as the reference methods.

3. Sensitivity analysis of time-varying inertia weight and acceleration coefficients to the order of nonlinear functions

In contrast to linear time-varying inertia weight and acceleration coefficients, in this study, nonlinear functions are proposed to adjust them. Non-linear high-order function has advantage to update PSO parameters. For instance, a faster reduction of the inertia weight can be achieved in the early stage, which will increase the rate of convergence. In the surrounding area of the optimum, the reduction of the inertia weight becomes slower, which will be helpful for capturing the solution.

$$w(k+1) = w_{\max} - (w_{\max} - w_{\min}) \left(\frac{k}{k_{\max}} \right)^{\alpha} \quad (6)$$

$$c_1(k+1) = c_{1i} - (c_{1i} - c_{1f}) \left(\frac{k}{k_{\max}} \right)^{\beta} \quad (7)$$

$$c_2(k+1) = c_{2i} - (c_{2i} - c_{2f}) \left(\frac{k}{k_{\max}} \right)^{\gamma} \quad (8)$$

Actually, constant and linear time-varying parameters are special cases of what are described by the above formulations. Inertia weight, cognitive and social parameters are constants when $\alpha = \beta = \gamma = 0$, and they are linearly time-varying parameters when $\alpha = \beta = \gamma = 1$.

To investigate the influences produced by α , β and γ on the performance of PSO, the values taken for them are listed in Table 1. In literature, commonly, a set of well-known nonlinear benchmark

Table 1
Test table of α , β and γ .

Name	Values
α	$0, \frac{1}{20}, \frac{1}{19}, \dots, \frac{1}{10}, \frac{1}{\pi^2}, \frac{1}{9}, \frac{1}{8}, \frac{1}{e^2}, \frac{1}{7}, \dots, \frac{1}{4}, \frac{1}{\pi}, \frac{1}{3}, \frac{1}{e}, \frac{1}{2}, 1, 2, e, 3, \pi, 4$
β	$0, \frac{1}{20}, \frac{1}{19}, \dots, \frac{1}{10}, \frac{1}{\pi^2}, \frac{1}{9}, \frac{1}{8}, \frac{1}{e^2}, \frac{1}{7}, \dots, \frac{1}{4}, \frac{1}{\pi}, \frac{1}{3}, \frac{1}{e}, \frac{1}{2}, 1, 2, e, 3, \pi, 4$
γ	$0, \frac{1}{5}, \frac{1}{4}, \frac{1}{\pi}, \frac{1}{3}, \frac{1}{e}, \frac{1}{2}, 1, 2, e, 3, \pi, 4, \dots, 9$

functions is used to test the performance of a new particle swarm optimization algorithm. The functions, search space of variables and their optimal values are given in Table 2. Sphere, Rotated hyper-ellipsoid and Ackley functions are used to assess the effectiveness of different values of α , β and γ . Sphere is an unimodal optimization problem. Rotated hyper-ellipsoid is an unimodal function with rotation. Ackley is a multimodal problem which is hard to optimize. These functions are tested in 20 dimensions; the population size for

all tests is 10 and $k_{\max}$ equals to 1000. The mean fitness value of the best particle found by 30 independent runs for the three functions are given in Tables 3–5.

Table 3 shows that the different values of α can result in quite different optimal solutions. For Sphere function, when the value of α changes within the range from $1/\pi^2$ to $1/18$, 1000 iterations can yield very good solutions. Similarly, better solutions of Rotated hyper-ellipsoid can be found when α varies from $1/9$ to $1/19$. The best solution of Ackley can be obtained when α equals to $1/\pi^2$. Generally, the optimal solutions generated from 1000 iterations for these three test functions are quite good considering the dimensions of variables and the sizes of particles when α is $1/\pi^2$. Overall, taking into account the solutions resulted from 500 iterations, which can reflect the convergence rate more or less, PSO with nonlinearly decreasing inertia weight adjusted by using the value $1/\pi^2$ for α has excellent performance.

Table 2
Benchmark test functions.

Function	Formula	Search space	Optimal value
Sphere	$f(x) = \sum_{i=1}^n x_i^2$	$[-100, 100]^n$	0
Weighted sphere	$f(x) = \sum_{i=1}^n ix_i^2$	$[-5.12, 5.12]^n$	0
Schwefel's	$f(x) = \sum_{i=1}^n \sum_{j=1}^i x_j^2$	$[-65.536, 65.536]^n$	0
Rotated hyper-ellipsoid	$f(x) = \sum_{i=1}^n \left(\sum_{j=1}^i x_j \right)^2$	$[-100, 100]^n$	0
Rosenbrock	$f(x) = \sum_{i=1}^{n-1} \left(100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2 \right)$	$[-5, 10]^n$	0
Easom	$f(x) = -\cos(x_1)\cos(x_2)\exp\left(-\left((x_1 - \pi)^2 + (x_2 - \pi)^2\right)\right)$	$[-100, 100]^n$	-1
Brain	$f(x) = \left(x_2 - \frac{5.1}{4\pi^2}x_1^2 + \frac{5.1}{\pi}x_1 - 6\right)^2 + 10\left(1 - \frac{1}{8\pi}\right)\cos(x_1) + 10$	$x_1 \in [-5, 10], x_2 \in [0, 15]$	0.397887
Rastrigin	$f(x) = \sum_{i=1}^n \left(x_i^2 - 10\cos(2\pi x_i) + 10\right)$	$[-5.12, 5.12]^n$	0
Ackley	$f(x) = -20\exp\left(-0.2\sqrt{\frac{1}{n}\sum_{i=1}^n x_i^2}\right) - \exp\left(\frac{1}{n}\sum_{i=1}^n \cos(2\pi x_i)\right) + 20 + e$	$[-32, 32]^n$	0
Griewank	$f(x) = \frac{1}{4000}\sum_{i=1}^n x_i^2 - \prod_{i=1}^n \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$	$[-600, 600]^n$	0

Table 3
The mean best solution found by different-order functions of inertia weight ($w_{\max} = 0.9$, $w_{\min} = 0.4$, $c_{1f} = c_{2f} = 2.0$).

α	Sphere		Rotated hyper-ellipsoid		Ackley	
	500 iterations	1000 iterations	500 iterations	1000 iterations	500 iterations	1000 iterations
4	17.41705	14.82121	1979.728	1457.640	2.584063	2.494667
π	17.86784	13.87858	2022.311	1593.941	2.561153	2.497753
3	17.27752	14.85113	1986.266	1337.311	2.542412	2.462076
e	16.73963	13.86454	1968.266	1576.012	2.576545	2.458632
2	17.09127	14.67176	1799.048	1294.794	2.651743	2.421623
1	16.65655	14.21571	2038.184	1409.665	2.558526	2.425273
$1/2$	13.20562	11.81127	1188.808	851.4957	2.414804	2.266351
$1/e$	8.401899	7.749229	642.8070	364.7371	1.995012	1.879774
$1/3$	7.202599	5.017506	384.4727	199.0917	1.929232	1.563818
$1/\pi$	6.879852	5.174911	317.2495	188.9493	1.696598	1.461526
$1/4$	3.836532	2.763512	104.8695	57.82326	1.603990	0.944415
$1/5$	1.786259	1.278156	18.76682	11.00864	0.662995	0.507386
$1/6$	0.664389	0.327879	3.279338	0.882052	0.307800	0.204043
$1/7$	0.263503	0.043912	0.472990	0.017194	0.187922	0.073711
$1/e^2$	0.125498	0.019574	503.9962	373.5520	0.127972	0.041123
$1/8$	0.089810	0.004005	0.047992	0.000636	0.082157	0.033765
$1/9$	0.024058	0.000667	0.005473	9.27E-06	0.629347	0.007089
$1/\pi^2$	0.008379	4.43E-05	0.002116	4.69E-08	1.422830	0.002556
$1/10$	0.010416	6.80E-05	0.002220	3.70E-08	0.510026	0.409797
$1/11$	0.008993	3.83E-06	0.000474	3.41E-09	1.012975	0.386012
$1/12$	0.002551	6.91E-07	0.000141	2.42E-11	2.483654	0.634767
$1/13$	0.001147	1.43E-07	3.76E-05	2.15E-11	2.192699	1.583203
$1/14$	0.420573	5.97E-08	0.000175	1.03E-14	3.29179	2.821922
$1/15$	0.004220	5.27E-09	1.58E-05	4.87E-11	2.962528	3.368703
$1/16$	0.082246	4.24E-09	90.45077	1.33E-13	4.750808	2.901271
$1/17$	2.856657	1.27E-08	2150.114	4.32E-11	6.075417	4.333301
$1/18$	2.201812	4.13E-09	39699.59	9.00E-10	3.642609	3.225652
$1/19$	26.20615	1.924455	329017.7	3.09E-12	7.545442	4.927587
$1/20$	93.57274	0.296829	1644662	9410.394	7.674822	5.428605
0	17.56220	14.81414	1984.125	1490.872	2.616007	2.444000

Table 4The mean best solution found by different-order functions of cognitive parameter ($w_{\max} = w_{\min} = 0.9$, $c_{1i} = 2.5$, $c_{1f} = 0.5$, $c_{2i} = c_{2f} = 2.0$).

β	Sphere		Rotated hyper-ellipsoid		Ackley	
	500 iterations	1000 iterations	500 iterations	1000 iterations	500 iterations	1000 iterations
4	19.89748	17.39257	3014.533	1639.083	2.736992	2.635723
π	18.82070	17.44129	2740.921	1776.667	2.750122	2.613078
3	20.61303	16.83421	2527.440	1891.113	2.718155	2.592954
e	18.86157	16.49192	3012.247	1842.537	2.685900	2.607001
2	18.49282	16.61823	2470.715	2004.972	2.748798	2.603959
1	19.91963	17.26305	2521.767	1972.893	2.689505	2.606053
1/2	20.03801	16.35450	2494.645	1759.740	2.708624	2.605795
1/e	18.42670	16.52352	2714.074	1676.280	2.682246	2.611616
1/3	17.81369	14.82731	2427.991	1715.151	2.679313	2.540620
1/ π	18.85147	15.57561	2211.064	1756.978	2.658311	2.470701
1/4	16.82192	13.94261	2286.755	1638.322	2.648974	2.483781
1/5	15.63209	13.94299	1864.353	1398.115	2.553377	2.453856
1/6	15.64688	13.31343	1761.781	1330.085	2.572133	2.383353
1/7	14.69972	13.01103	1388.993	1035.340	2.508505	2.313791
1/e ²	14.29867	11.74921	2648.401	1985.769	2.456404	2.275866
1/8	14.31140	12.04774	1504.954	1101.640	2.457696	2.299622
1/9	12.79718	11.16063	1202.505	812.0013	2.445074	2.236730
1/ π^2	13.14489	10.26753	1247.080	717.8869	2.482761	2.066535
1/10	12.27982	10.04800	1118.960	801.2830	2.419000	2.143918
1/11	14.76872	10.06167	1243.037	593.6385	2.309689	2.208628
1/12	11.74059	9.063453	1135.955	656.4074	2.339268	2.047364
1/13	12.60066	8.933728	1072.070	536.7220	2.341209	1.981017
1/14	10.93264	9.101819	1091.703	518.3832	2.313319	2.099919
1/15	12.76481	7.857574	1065.296	482.1457	2.247239	1.882562
1/16	11.65409	7.444044	894.0729	441.6738	2.203056	1.912603
1/17	10.89844	8.790892	898.3352	409.2332	2.148792	1.955877
1/18	11.08553	8.078215	919.1678	465.4155	2.117819	1.736648
1/19	12.51415	8.709060	949.9734	376.1163	2.180685	1.823021
1/20	11.83444	8.051571	879.5410	365.9437	2.221166	1.938976
0	17.56220	14.81414	1984.125	1490.872	2.616007	2.444000

From Tables 4 and 5, it can be easily observed that there are no big differences between the best solutions for all the three functions even when the values of β or γ are obviously different, in another word, the performance of a PSO algorithm is just slightly dependent on the changes of cognitive and social parameters during the iteration process. The main reason is that the effects of the values of these two parameters on the velocity of the particles, as shown in Eq. (1), are dependent on the values of random numbers which are uniformly distributed in the interval $[0, 1]$. It is the stochastic characteristics of PSO algorithms that the random numbers can change the velocities and positions of particles randomly.

In Table 3, the mean best solution for Sphere, Rotated hyper-ellipsoid and Ackley corresponding to $\alpha = 1/\pi^2$ is respectively

$4.43E - 5$, $4.69E - 8$ and 0.002556 when the iterations are 1000. The best optimum for these three benchmark optimization problems is respectively, 7.444044, 365.9437 and 1.736648 in Table 4, and 2.209919, 32.02607 and 1.12324 in Table 5, under the same iterations. Comparing Tables 4 and 5 and with Table 3, it can be concluded that the change of α produces much greater effect on the PSO performance than β and γ . PSO performance is much more sensitive to the order of nonlinear function which is used to change the inertia weight. When a proper order of the nonlinear function for inertia weight is determined, for example, α takes $1/\pi^2$, the contributions made by the time-varying cognitive and social parameters can be ignored. Therefore, the two acceleration coefficients can be treated as constants.

Table 5The mean best solution found by different-order functions of social parameter ($w_{\max} = w_{\min} = 0.9$, $c_{1i} = c_{1f} = 2.0$, $c_{2i} = 0.5$, $c_{2f} = 2.5$).

γ	Sphere		Rotated hyper-ellipsoid		Ackley	
	500 iterations	1000 iterations	500 iterations	1000 iterations	500 iterations	1000 iterations
9	3.453098	3.156070	151.5699	68.21564	1.511632	1.185145
8	3.453098	3.156070	87.07481	61.73942	1.511632	1.185145
7	3.453098	3.156070	103.6741	59.28555	1.511632	1.185145
6	3.453098	3.156070	125.3053	58.08141	1.511632	1.185145
5	3.453098	3.156070	157.4597	59.74140	1.511632	1.185145
4	3.453098	3.156070	82.50913	47.96449	1.511632	1.185145
π	4.371490	2.209919	93.18796	32.02607	1.511622	1.185156
3	3.453092	3.156082	114.4508	92.09470	1.511624	1.190316
e	3.453062	3.156153	120.4491	84.62423	1.704614	1.123240
2	3.018651	3.056063	90.91154	78.07262	1.464621	1.222143
1	3.262646	3.112382	92.53567	74.89379	1.531950	1.367145
1/2	5.104867	3.842852	212.5444	107.1596	1.692304	1.386957
1/e	6.062271	4.656080	273.7789	132.6110	1.675190	1.385797
1/3	6.817229	4.897822	339.9748	167.1807	1.678224	1.450264
1/ π	7.458635	4.461149	327.1599	199.2815	1.830466	1.385166
1/4	8.890188	5.543985	457.0213	257.7683	1.843213	1.591847
1/5	8.124088	6.512405	508.4738	297.3340	2.166461	1.591246
0	17.56220	14.81414	1984.125	1490.872	2.616007	2.444000

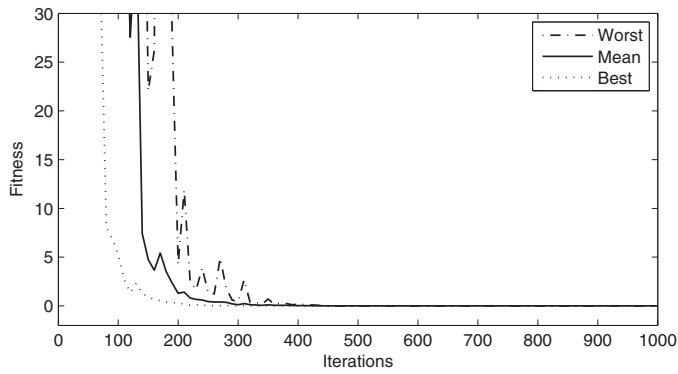


Fig. 1. Worst, mean and best fitness of Sphere function for population size 10.

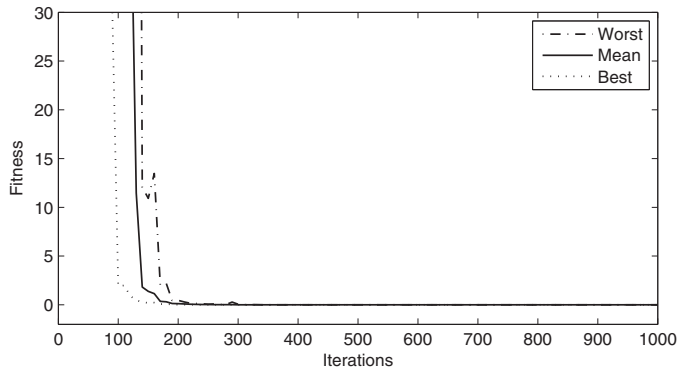


Fig. 2. Worst, mean and best fitness of Sphere function for population size 30.

Based on the above conclusion, the performance of our improved PSO with $\alpha = 1/\pi^2$, $w_{\max} = 0.9$, $w_{\min} = 0.4$, $\beta = \gamma = 0$ and $c_{1i} = c_{1f} = c_{2i} = c_{2f} = 2.0$ are further tested when different population sizes (10, 30 and 50) are adopted. The worst, mean and best fitness values found by 30 runs versus iterations for Sphere, Rotated hyper-ellipsoid and Ackley functions are shown in Figs. 1–9. The comparison of mean fitness convergence speed between different population sizes for these three test functions can be seen in Figs. 10–12. We sample the data at every 10 iterations to 400 iterations and every 50 iterations afterwards to plot these figures. In order to clearly demonstrate the convergence trend, all Figs. 1–12 only capture the most important parts of the whole convergence diagrams.

All above figures explicitly illustrate that haphazard increasing population size may not improve the convergence speed when the population size has reached a certain number (that is 10 here). Drastic improvement cannot be observed from the results for Sphere

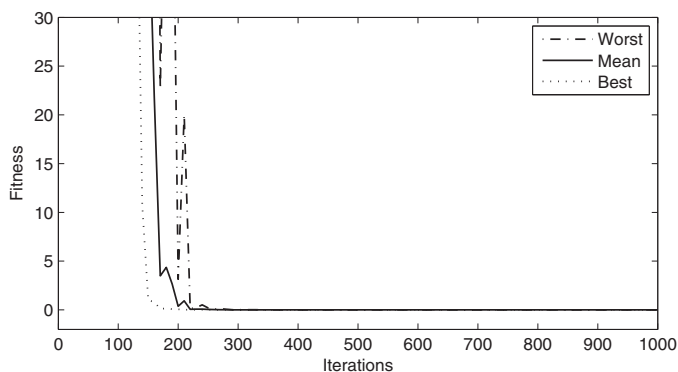


Fig. 3. Worst, mean and best fitness of Sphere function for population size 50.

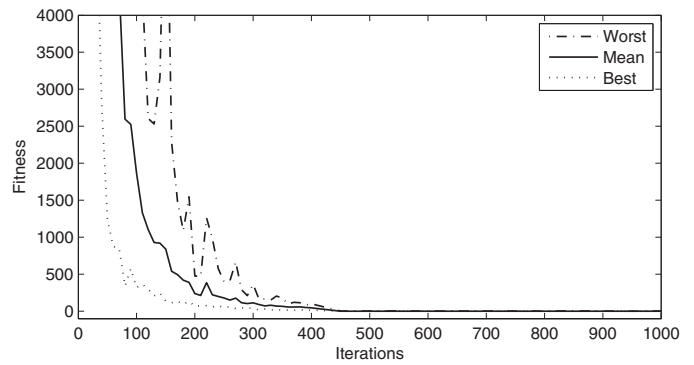


Fig. 4. Worst, mean and best fitness of Rotated hyper-ellipsoid function for population size 10.

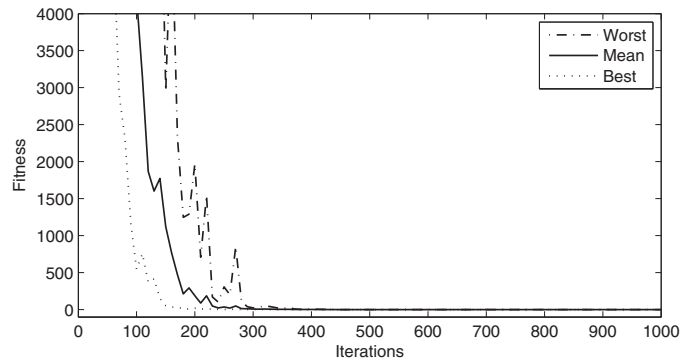


Fig. 5. Worst, mean and best fitness of Rotated hyper-ellipsoid function for population size 30.

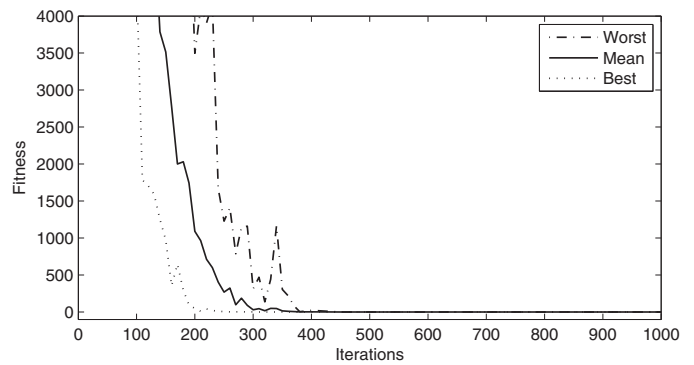


Fig. 6. Worst, mean and best fitness of Rotated hyper-ellipsoid function for population size 50.

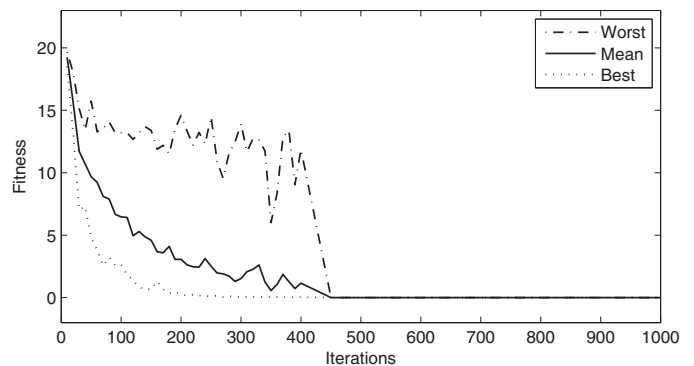


Fig. 7. Worst, mean and best fitness of Ackley function for population size 10.

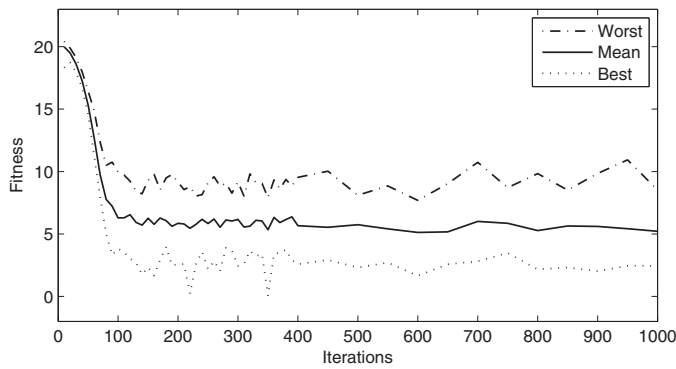


Fig. 8. Worst, mean and best fitness of Ackley function for population size 30.

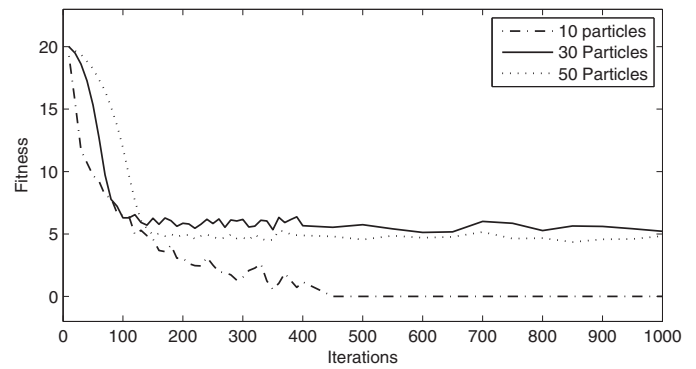


Fig. 12. Mean fitness of Ackley function for population size 10, 30 and 50.

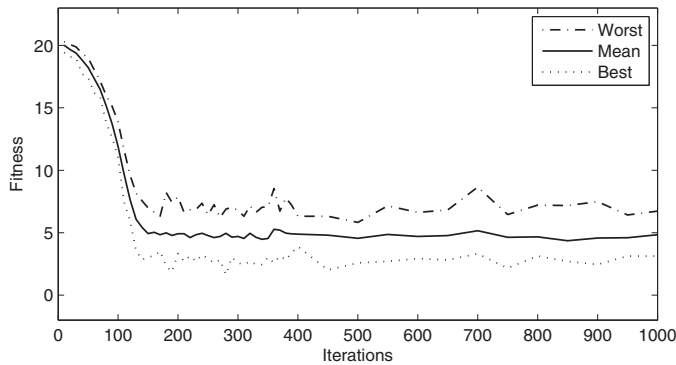


Fig. 9. Worst, mean and best fitness of Ackley function for population size 50.

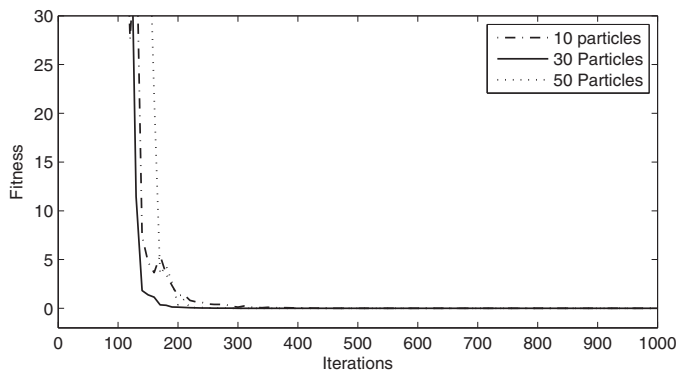


Fig. 10. Mean fitness of Sphere function for population size 10, 30 and 50.

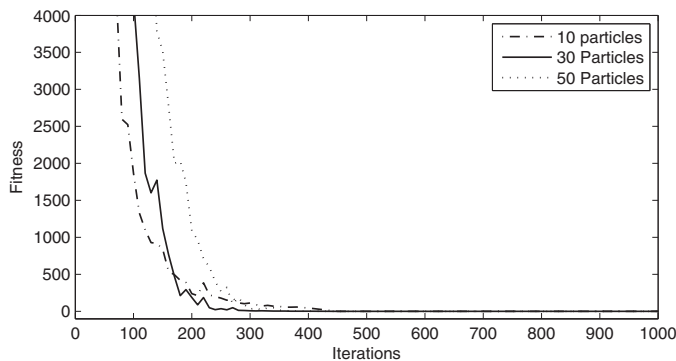


Fig. 11. Mean fitness of Rotated hyper-ellipsoid function for population size 10, 30 and 50.

and Rotated hyper-ellipsoid functions. These two functions are continuous, convex and unimodal, so our improved PSO does not require many particles to capture the global optimal. Sometimes, for Ackley function, a larger population size may hinder the convergence and accuracy of PSO. Although it has been concluded that a larger population size should be more effective on functions with more numerous and dramatic local optima, the authors also pointed out that PSO using the star sociometry sometimes converges quickly to a suboptimal point in the early stage instead of thoroughly exploring the whole space [36]. In order to further investigate this phenomenon, Ackley function in 2 dimensions is tested again when population sizes 10, 30 and 50 are adopted. Our improved PSO is able to converge to the global optimal with different population sizes, shown in Fig. 13. It can be therefore concluded that it has higher risk to stuck to a suboptimal region with a larger population size adopted when only the star sociometry is used. The population size from 25 to 60 for Ackley function in 30 dimensions is recommended in [37], but they used dynamic sociometry which is different with this paper. In the future, the stability of the presented PSO algorithm can be further improved by using dynamic topology.

4. LHNSO and computational results

Here, we propose a low-discrepancy sequence initialized particle swarm optimization algorithm with high-order nonlinear time-varying inertia weight (LHNSO). The initial population of particles is generated by using the Halton sequence to cover the search space efficiently. It has to be noted, based on our experience, low-discrepancy sequence initialization will not greatly improve the performance of our proposed method on these benchmark test functions, but low-discrepancy sampling based methods have been widely adopted to solve various real world problems and

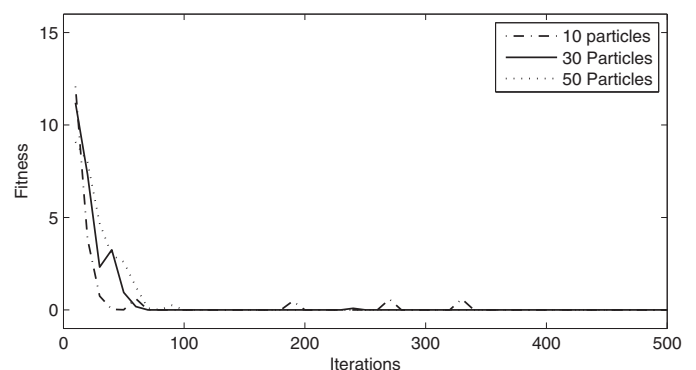


Fig. 13. Mean fitness of Ackley function in two dimensions for population size 10, 30 and 50.

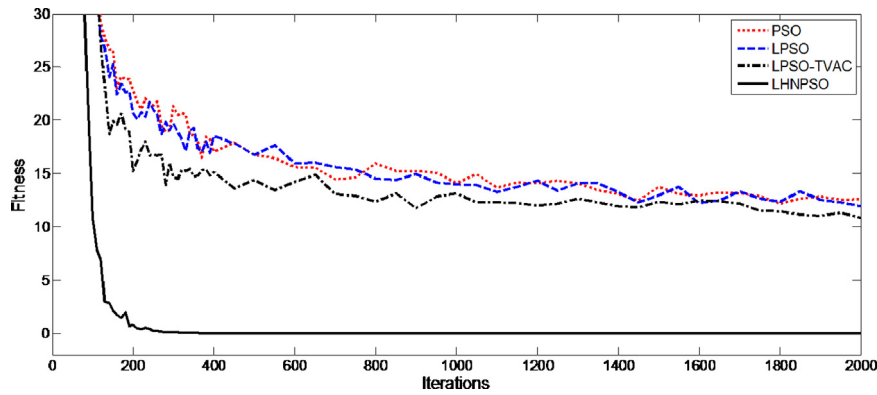


Fig. 14. Mean fitness of Sphere function for PSO, LPSO, LPSO-TVAC and LHNPSO.

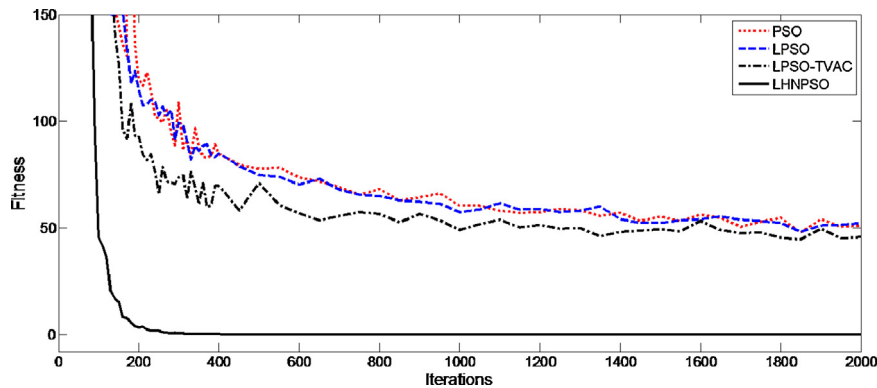


Fig. 15. Mean fitness of Schwefel's function for PSO, LPSO, LPSO-TVAC and LHNPSO.

the increased computational burden can be neglected [33,38,39]; therefore, keeping it is able to ensure the wide implementation of LHNPSO. In addition, inertial weight is varied according to the nonlinear function given in Eq. (8) with $\alpha = 1/\pi^2$, $w_{\max} = 0.9$ and $w_{\min} = 0.4$. As the contributions made by the changes of the two acceleration coefficients are not significant, the two acceleration coefficients, cognitive parameter and social parameter are taken as constant values, that is, $\beta = \gamma = 0$ and $c_{1i} = c_{1f} = c_{2i} = c_{2f} = 2.0$.

To evaluate the performance of the proposed LHNPSO algorithm, the classical PSO, LPSO and LPSO-TVAC are used for comparisons. In the classical PSO, $\alpha = \beta = \gamma = 0$, $w_{\max} = w_{\min} = 0.9$ and $c_{1i} = c_{1f} = c_{2i} = c_{2f} = 2.0$. In the LPSO, $\alpha = 1$, $w_{\max} = 0.9$, $w_{\min} = 0.4$ and $c_{1i} = c_{1f} = c_{2i} = c_{2f} = 2.0$. In the LPSO-TVAC, $\alpha = \beta = \gamma = 1$, $w_{\max} = 0.9$, $w_{\min} = 0.4$, $c_{1i} = c_{2f} = 2.5$ and $c_{1f} = c_{2i} = 0.5$. Except the two-dimensional functions Easom and Brain, other benchmark

problems are tested in 20 dimensions. The population size for all tests is 10. The mean fitness values of the best particle found by 30 runs for the ten functions are given in Table 6, while the iterations are 1000. To illustrate the convergence speed, the mean fitness values of the best particle found by 30 independent runs versus the iterations for functions Sphere, Schwefel's, Rotated hyper-ellipsoid, Rosenbrock and Ackley are shown in Figs. 14–18. Similar to Figs. 1–12, Figs. 14–18 also capture the most bottom parts of the whole convergence diagrams.

Table 6 shows that all the four PSO algorithms can find very good solutions for Brain function which is a simple two dimensional problem. PSO, LPSO and the proposed LHNPSO can get much more accurate results for Easom function than LPSO-TVAC using 1000 iterations. It should be noted that the time-varying acceleration coefficients may not improve the PSO performance for some

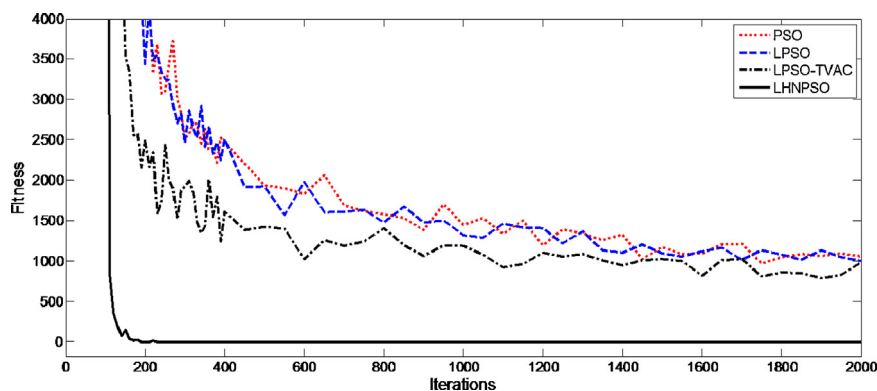
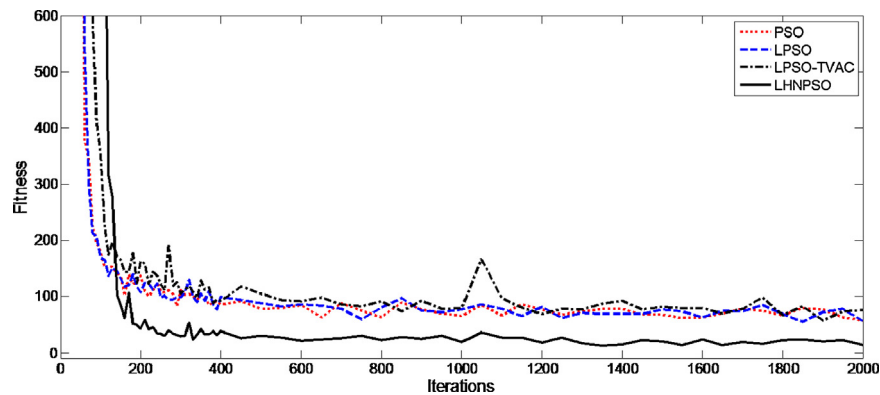
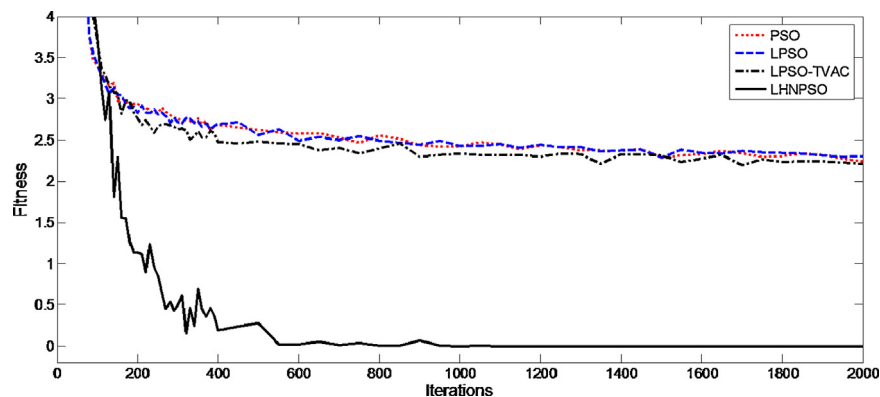


Fig. 16. Mean fitness of rotated hyper-ellipsoid function for PSO, LPSO, LPSO-TVAC and LHNPSO.

Table 6

The mean best solution found by PSO, LPSO, LPSO-TVAC, LHNPSO.

Function	PSO	LPSO	LPSO-TVAC	LHNPSO
Sphere	14.81415	14.51174	11.88231	3.64E–05
Weighted sphere	0.3720585	0.3403115	0.3137708	1.69E–05
Schwefel's	62.4754	60.89549	49.62875	0.000252
Rotated hyper-ellipsoid	1533.51	1480.281	1081.471	4.28E–07
Rosenbrock	87.07232	76.81533	85.6209	11.32098
Easom	–0.999697566	–0.999838867	–0.764122515	–1
Brain	0.397891	0.397888	0.397888	0.397887
Rastrigin	37.82494	40.02787	48.13907	24.54232
Ackley	2.444000	2.515411	2.362284	0.001681
Griewank	1.132407	1.13347	1.105666	0.03682

**Fig. 17.** Mean fitness of Rosenbrock function for PSO, LPSO, LPSO-TVAC and LHNPSO.**Fig. 18.** Mean fitness of Ackley function for PSO, LPSO, LPSO-TVAC and LHNPSO.

optimization problems as the similar phenomenal can be found in Fig. 17 for function Rosenbrock. As what is explained in Section 3, the contributions of the time-varying cognitive and social parameters are largely affected by the random numbers. For other eight benchmark test functions, LHNPSO generates significantly better solutions than PSO, LPSO and LPSO-TVAC.

Figs. 14–18 show that LHNPSO converges much faster for all the five functions than the classical PSO, LPSO and LPSO-TVAC. LPSO-TVAC does show the worst performance for function Rosenbrock. It is clear that LHNPSO converges quickly to the global/real optima for all the five benchmark problems, however, the classical PSO, LPSO and LPSO-TVAC hardly hit the optima, which exhibits obvious prematurity and may fall into the local optima. The proposed LHNPSO can effectively prevent the premature convergence and significantly improve the convergence rate and accuracy. In summary, for all the test functions, the proposed LHNPSO shows a better performance than the classical PSO, LPSO and LPSO-TVAC.

5. Conclusions

A new variant of PSO is proposed which employs the low-discrepancy sequence and decreasing inertia weight adjusted by high-order $(1/\pi^2)$ nonlinear function. The performance of LHNPSO algorithm is tested by a set of well-known benchmark test functions and the results show that this method can converge faster and give a much more accurate final solution in comparison with PSO, LPSO and LPSO-TVAC algorithms. Moreover, the proposed LHNPSO is very easily to be implemented. From this study, it can also be obtained that the performance of PSO algorithm is not significantly affected by the time-varying cognitive and social parameters. Sometimes, changing acceleration coefficients even impairs PSO performance. In this paper, only the unconstrained optimization problems are used to test the performance of LHNPSO. In the future, LHNPSO will be further developed to solve more complex problems.

Acknowledgements

The authors would like to thank the reviewers for constructive comments on an earlier version of the paper. This research was supported by the Australian Research Council through ARC Discovery Project 130102934.

References

- [1] J. Kennedy, R. Eberhart, Particle swarm optimization, *Proc. IEEE Int. Conf. Neural Netw.* 4 (1995) 1942–1948.
- [2] A. Nickabadi, M.M. Ebadzadeh, R. Safabakhsh, A novel particle swarm optimization algorithm with adaptive inertia weight, *Appl. Soft Comput.* 11 (4) (2011) 3658–3670.
- [3] B. Akay, A study on particle swarm optimization and artificial bee colony algorithms for multilevel thresholding, *Appl. Soft Comput.* 13 (6) (2013) 3066–3091.
- [4] B. Bolat, O. Altun, P. Corts, A particle swarm optimization algorithm for optimal car-call allocation in elevator group control systems, *Appl. Soft Comput.* 13 (5) (2013) 2633–2642.
- [5] Y. Marinakis, G.R. Iordanidou, M. Marinaki, M. Marinaki, Particle swarm optimization for the vehicle routing problem with stochastic demands, *Appl. Soft Comput.* 13 (4) (2013) 1693–1704.
- [6] F. Kang, J.J. Li, Q. Xu, Damage detection based on improved particle swarm optimization using vibration data, *Appl. Soft Comput.* 12 (8) (2012) 2329–2335.
- [7] S. Yang, C. Li, A clustering particle swarm optimizer for locating and tracking multiple optima in dynamic environments, *IEEE Trans. Evol. Comput.* 14 (6) (2010) 959–974.
- [8] Q. Wu, Product demand forecasts using wavelet kernel support vector machine and particle swarm optimization in manufacture system, *J. Comput. Appl. Math.* 233 (10) (2010) 2481–2491.
- [9] C.J. Liao, P. Luarn, A discrete version of particle swarm optimization for flow-shop scheduling problems, *Comput. Oper. Res.* 34 (10) (2007) 3099–3111.
- [10] C. Xing, Y. Li, K. Zhang, L. Wang, Shadow detecting using particle swarm optimization and the Kolmogorov test, *Comput. Math. Appl.* 62 (7) (2011) 2704–2711.
- [11] X. Fu, A. Li, L. Wang, C. Ji, Short-term scheduling of cascade reservoirs using an immune algorithm-based particle swarm optimization, *Comput. Math. Appl.* 62 (6) (2011) 2463–2471.
- [12] I. Montalvo, J. Izquierdo, R. Pérez, M.M. Tung, Particle swarm optimization applied to the design of water supply systems, *Comput. Math. Appl.* 56 (3) (2008) 769–776.
- [13] Y. Gao, G. Zhang, J. Lu, H.-M. Wee, Particle swarm optimization for bi-level pricing problems in supply chains, *J. Glob. Optim.* 51 (2) (2010) 245–254.
- [14] M.M. Noel, A new gradient based particle swarm optimization algorithm for accurate computation of global minimum, *Appl. Soft Comput.* 12 (1) (2012) 353–359.
- [15] M.R. Chen, X. Li, X. Zhang, Y.Z. Lu, A novel particle swarm optimizer hybridized with extremal optimization, *Appl. Soft Comput.* 10 (2) (2010) 367–373.
- [16] N.M. Kwok, Q.P. Ha, T.H. Nguyen, J. Li, B. Samali, A novel hysteretic model for magnetorheological fluid dampers and parameter identification using particle swarm optimization, *Sens. Actuators A: Phys.* 132 (2) (2006) 441–451.
- [17] V. Plevris, M. Papadrakakis, A hybrid particle swarm-gradient algorithm for global structural optimization, *Comput.-Aided Civil Infrastruct. Eng.* 26 (1) (2011) 48–68.
- [18] R. Akbari, K. Ziarati, A rank based particle swarm optimization algorithm with dynamic adaptation, *J. Comput. Appl. Math.* 235 (8) (2011) 2694–2714.
- [19] W.F. Abd-El-Wahed, A.A. Mousa, M.A. El-Shorbagy, Integrating particle swarm optimization with genetic algorithms for solving nonlinear optimization problems, *J. Comput. Appl. Math.* 35 (5) (2011) 1446–1453.
- [20] Y. Tang, Z. Wang, J. Fang, Feedback learning particle swarm optimization, *Appl. Soft Comput.* 11 (8) (2011) 4713–4725.
- [21] R.J. Kuo, Y.S. Han, A hybrid of genetic algorithm and particle swarm optimization for solving bi-level linear programming problem – a case study on supply chain model, *Appl. Math. Model.* 35 (8) (2011) 3905–3917.
- [22] H. Lu, W. Chen, Self-adaptive velocity particle swarm optimization for solving constrained optimization problems, *J. Glob. Optim.* 41 (3) (2007) 427–445.
- [23] R.C. Eberhart, R.C. Shi, Tracking and optimizing dynamic systems with particle swarms, in: *Proceedings of the IEEE Conference on Evolutionary Computation, ICEC*, vol. 1, 2001, pp. 94–100.
- [24] Y. Shi, R.C. Eberhart, Empirical study of particle swarm optimization, in: *Proceedings of the 1999 Congress on Evolutionary Computation-CEC99*, vol. 3, 1999, pp. 1945–1950.
- [25] A. Chatterjee, P. Siarry, Nonlinear inertia weight variation for dynamic adaptation in particle swarm optimization, *Comput. Oper. Res.* 33 (3) (2006) 859–871.
- [26] S.K.S. Fan, Y.Y. Chiu, A decreasing inertia weight particle swarm optimizer, *Eng. Optim.* 39 (2) (2007) 203–228.
- [27] P.K. Tripathi, S. Bandyopadhyay, S.K. Pal, Multi-Objective Particle Swarm Optimization with time variant inertia and acceleration coefficients, *Inf. Sci.* 177 (22) (2007) 5033–5049.
- [28] Q. Wu, Hybrid forecasting model based on support vector machine and particle swarm optimization with adaptive and Cauchy mutation, *Expert Syst. Appl.* 38 (8) (2011) 9070–9075.
- [29] C. Li, J. Zhou, P. Kou, J. Xiao, A novel chaotic particle swarm optimization based fuzzy clustering algorithm, *Neurocomputing* 83 (2012) 98–109.
- [30] X. Wang, A new measure of irregularity of distribution and quasi-Monte Carlo methods for global optimization, *Comput. Math. Appl.* 43 (6–7) (2002) 657–669.
- [31] G. Lei, Adaptive random search in quasi-Monte Carlo methods for global optimization, *Comput. Math. Appl.* 43 (6–7) (2002) 747–754.
- [32] A. Alessandri, C. Cervellera, D. Macciò, M. Sanguineti, Optimization based on quasi-Monte Carlo sampling to design state estimators for non-linear systems, *Optimization* 59 (7) (2010) 963–984.
- [33] W. Gao, D. Wu, C. Song, F. Tin-Loi, X. Li, Hybrid probabilistic interval analysis of bar structures with uncertainty using a mixed perturbation Monte-Carlo method, *Finite Elem. Anal. Des.* 47 (7) (2011) 643–652.
- [34] Y. Shi, R. Eberhart, A modified particle swarm optimizer, in: *1998 IEEE International Conference on Evolutionary Computation Proceedings, IEEE World Congress on Computational Intelligence*, 1998, pp. 69–73.
- [35] A. Ratnaweera, S.K. Halgamuge, H.C. Watson, Self-organizing hierarchical particle swarm optimizer with time-varying acceleration coefficients, *IEEE Trans. Evol. Comput.* 8 (3) (2004) 240–255.
- [36] M. Richards, D. Ventura, Dynamic Sociometry in particle swarm optimization, in: *Proceedings of the Joint Conference on Information Sciences*, 2003, pp. 1557–1560.
- [37] A. Mohais, R. Mendes, C. Ward, C. Posthoff, Neighborhood re-structuring in particle swarm optimization, in: *AI 2005: Advances in Artificial Intelligence*, Springer, Berlin, Heidelberg, 2005.
- [38] N. Liu, W. Gao, C. Song, N. Zhang, Y.-L. Pi, Interval dynamic response analysis of vehicle–bridge interaction system with uncertainty, *J. Sound Vib.* 332 (13) (2013) 3218–3231.
- [39] D.M. Do, W. Gao, C.W. Yang, C.M. Song, Dynamic analysis of structures with interval parameters under stochastic process excitation, *Appl. Mech. Mater.* 553 (2014) 699–704.